DESIGN PROJECT – II · SEMESTER IV

PhishLens

Explainable Hybrid Phishing-URL Detection with Out-of-Template Evaluation

A project report submitted in partial fulfilment of the requirements of
Design Project – II (Semester IV) for the degree of

BACHELOR OF TECHNOLOGY

with specialisation in Cyber Security

Submitted by

Malepati Sree Harsha

Register No. 23CU0340018

DISC

HINDUSTAN INSTITUTE OF TECHNOLOGY AND SCIENCE

(Deemed to be University)

Padur, Kelambakkam, Chennai – 603 103

Academic Year 2024–25

DECLARATION

I, **Malepati Sree Harsha** (Register No. 23CU0340018), hereby declare that the project report entitled **“PhishLens: Explainable Hybrid Phishing-URL Detection with Out-of-Template Evaluation”**, submitted as Design Project – II in Semester IV of the Bachelor of Technology programme with specialisation in Cyber Security, DISC, Hindustan Institute of Technology and Science, during the academic year 2024–25, is a record of my own work.

Where the work of others has been used, it is acknowledged and cited in the references. All experiments were performed on systems I own, using bundled or synthetic data, and no third-party system was tested without authorization.

Place: Chennai

Date: _____

Malepati Sree Harsha

Register No. 23CU0340018

ACKNOWLEDGEMENT

I thank the faculty of DISC, Hindustan Institute of Technology and Science, for the course structure, laboratory facilities and feedback that shaped this design project, and the institution for the opportunity to work on an applied security problem.

I also acknowledge the open-source communities behind Python, Flask, scikit-learn, the cryptography library, OpenSSL, John the Ripper, Wireshark/TShark, Suricata, Nmap and OWASP ZAP, whose tools and documentation made this work possible. Finally, I thank my family and friends for their support.

Malepati Sree Harsha

ABSTRACT

Phishing remains one of the most common initial-access techniques, and machine-learning URL classifiers frequently report accuracies above 95%. Such figures are usually measured on test data drawn from the same distribution as the training data, and the models rarely explain their decisions. This project, PhishLens, is a local web application that scores URLs, pasted e-mails and batches of links with an explainable logistic-regression model over 16 interpretable URL features, shows each feature's contribution to the score, and offers opt-in, authorization-gated passive enrichment (WHOIS, DNS SPF/DMARC records and TLS certificate details).

Beyond building the tool, the project evaluates it honestly. On a synthetic in-distribution test set of 2,400 URLs, four classifiers (logistic regression, decision tree, random forest, Gaussian naive Bayes) tie at about 97% F1, the ceiling set by 3% deliberate label noise. However, on hand-written **out-of-template probes** that use tricks absent from the training generator (typosquats, combosquats, punycode on common TLDs), the model alone detected only 1 of 10 phishing URLs, a clear case of shortcut learning. A **hybrid scorer** that adds typosquat and combosquat signals and lets high-severity rule signals set a risk floor raised hold-out recall to 9/10 with 0/12 false positives, on a probe set fixed before the hybrid was designed. The work also found and fixed a server-side request forgery (SSRF) weakness in the TLS enrichment path. The system is containerised, hardened with OWASP headers and rate limits, and verified by 18 automated tests.

Keywords: phishing detection, explainable machine learning, logistic regression, typosquatting, combosquatting, homograph attack, out-of-distribution evaluation, SSRF, Flask, Docker

CONTENTS

Declaration ii
Acknowledgement iii
Abstract iv
List of Figures vii
List of Tables viii
Chapter 1 Introduction 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 1
1.4 Objectives 1
1.5 Scope and Ethics 2
1.6 Organisation of the Report 2
Chapter 2 Literature Survey 3
2.1 Blocklists and Reputation 3
2.2 URL-Feature Classifiers 3
2.3 Domain Squatting and Homographs 3
2.4 Explainability and Shortcut Learning 3
2.5 Comparison and Research Gap 3
Chapter 3 System Requirements Specification 5
3.1 Functional Requirements 5
3.2 Non-Functional Requirements 5
3.3 Hardware and Software Requirements 5
Chapter 4 System Design 7
4.1 Architecture 7
4.2 Data Flow Diagrams 8
4.3 UML Diagrams 9
4.4 Deployment 11
4.5 Feature Set 11
Chapter 5 Methodology and Implementation 12
5.1 Technology Stack 12
5.2 Stage 1 — Feature Extraction and Rule Signals 12
5.3 Stage 2 — Synthetic Training Data 12
5.4 Stage 3 — Explainable Model 12
5.5 Stage 4 — Model Comparison 13
5.6 Stage 5 — Out-of-Template Probes (Pre-Registered Hold-Out) 13
5.7 Stage 6 — Typosquat, Combosquat and the Hybrid Floor 13

5.8 Stage 7 — Safe Passive Enrichment 14
5.9 Stage 8 — Hardening, Logging and Performance 15
5.10 Secure Coding Practices Applied 15
Chapter 6 Testing and Validation 16
6.1 Strategy 16
6.2 Automated Test Cases 16
6.3 Functional Checks on Sample URLs 17
Chapter 7 Results and Discussion 19
7.1 Working System 19
7.2 In-Distribution Model Comparison 20
7.3 Out-of-Template Generalisation 22
7.4 Security Finding: SSRF in Enrichment 23
7.5 Performance 23
7.6 Mobile Layout 24
7.7 Limitations 24
Chapter 8 Conclusion and Future Scope 25
8.1 Conclusion 25
8.2 Future Scope 25
References 26
Appendix A Installation and User Manual 28
Appendix B Demonstration Script 29
Appendix C API Reference 30
Appendix D Repository Structure 31

LIST OF FIGURES

Figure 4.1 Layered architecture of PhishLens 7
Figure 4.2 DFD level 0 (context diagram) 8
Figure 4.3 DFD level 1 8
Figure 4.4 Use-case diagram 9
Figure 4.5 Sequence diagram for analysing a pasted e-mail 9
Figure 4.6 Class and module diagram 10
Figure 4.7 Activity diagram of hybrid risk scoring 10
Figure 4.8 Deployment diagram 11
Figure 7.1 PhishLens analyser 19
Figure 7.2 Phishing verdict for a pasted e-mail with signals and feature drivers 19
Figure 7.3 Low-risk verdict for a legitimate URL 20
Figure 7.4 Batch analysis of seven URLs 20
Figure 7.5 F1 and ROC-AUC per model (axis zoomed) 21
Figure 7.6 Confusion matrix of the deployed logistic-regression model 21
Figure 7.7 Recall on out-of-template phishing probes 22
Figure 7.8 Punycode homograph: model risk is low, the hybrid floor raises the verdict 22
Figure 7.9 Latency per endpoint 23
Figure 7.10 Layout at 390 px width 24

LIST OF TABLES

Table 2.1 Existing approaches compared with PhishLens 3
Table 3.1 Functional requirements 5
Table 3.2 Non-functional requirements 5
Table 3.3 Hardware and software requirements 5
Table 4.1 The 16 model features 11
Table 5.1 Technology choices 12
Listing 5.2 Scoring with contributions and the hybrid floor (phishlens/model.py) 12
Listing 5.3 Squatting detection (phishlens/features.py) 14
Listing 5.4 SSRF-safe resolution (phishlens/tools.py) 14
Table 6.1 Test levels 16
Table 6.2 Automated test cases and results 16
Table 6.3 Live scoring of sample inputs (Docker build) 17
Table 7.1 Model comparison (train n = 2,400, test n = 2,400) 20
Table 7.2 Probe results (flagged = score ≥ 40) 22
Table 7.3 Load-test results (200 requests, 8 workers) 23

CHAPTER 1

INTRODUCTION

1.1 Background

Phishing persuades a user to visit a fraudulent site or reveal credentials by imitating a trusted brand. Industry reports consistently rank phishing and stolen credentials among the most frequent breach vectors [1], [2]. Defences operate at several layers: e-mail authentication (SPF, DMARC [14], [15]), blocklists of known-bad URLs [16], and classifiers that judge a URL or page by its features [3], [5], [18].

URL-based classifiers are attractive because they work before a page is loaded and need no network access. Ma et al. [3] showed that lexical and host-based URL features support accurate detection; many later studies report accuracies above 95% on their datasets [5].

1.2 Motivation

Two weaknesses limit how far such results can be trusted. First, most classifiers act as black boxes: an analyst sees “phishing, 0.93” but not *why*, so mistakes cannot be challenged. Second, high scores are usually measured on test data from the same distribution as the training data. A model can then learn shortcuts [13], features that correlate with the label in the dataset but not in the world, and fail silently on new attack styles. A learning project in this area should therefore build a detector, and also ask how it fails and make that failure visible.

1.3 Problem Statement

To build a local, explainable phishing-URL analyser whose every verdict can be traced to concrete signals and feature contributions; to measure not only in-distribution accuracy but also generalisation to attack styles absent from its training data; and to improve that generalisation without sacrificing explainability or safety.

1.4 Objectives

1. Extract 16 interpretable URL features and human-readable rule signals from URLs and e-mail bodies.
2. Train a standardised logistic-regression model and expose per-feature contributions for every prediction.

3. Compare it against decision tree, random forest and naive Bayes models with held-out tests and 5-fold cross-validation.
4. Design out-of-template probe sets, including a hold-out set fixed in advance, to measure generalisation honestly.
5. Add typosquat and combosquat detection and a hybrid scoring rule, and quantify their effect.
6. Provide opt-in, authorization-gated passive enrichment (WHOIS, DNS, TLS) that cannot be abused for SSRF.
7. Harden, containerise and test the system; measure its performance.

1.5 Scope and Ethics

PhishLens never opens or fetches the URLs it analyses. Enrichment runs only after the user confirms they own or are authorized to check the domain, and accepts only public DNS names. The training data are synthetic; probe URLs are illustrative strings using made-up domains and documentation IP ranges. No phishing site was visited and no third-party system was tested.

1.6 Organisation of the Report

Chapter 2 reviews related work and the research gap. Chapter 3 specifies requirements; Chapter 4 the design; Chapter 5 the implementation in stages. Chapter 6 covers testing, Chapter 7 results including the generalisation study, and Chapter 8 concludes.

CHAPTER 2

LITERATURE SURVEY

2.1 Blocklists and Reputation

Services such as Google Safe Browsing [16] and community feeds maintain lists of known phishing URLs. They have very low false-positive rates but cannot flag a URL until it has been reported, and phishing domains are often short-lived.

2.2 URL-Feature Classifiers

Garera et al. [4] identified structural URL patterns used by phishers (obfuscated hosts, brand names in paths, IP hosts). Ma et al. [3] trained classifiers on lexical and host-based features at scale. Sahingoz et al. [5] compared several algorithms on NLP-derived URL features and reported high accuracy. Content-based approaches such as CANTINA [18] instead analyse page text, at the cost of fetching the page.

2.3 Domain Squatting and Homographs

Typosquatting registers misspellings of popular domains [7]. Combosquatting joins a brand with other words, e.g. *brand-security-update.com*; Kintis et al. [6] found it far more common than typosquatting and longer-lived. Homograph attacks [8] use look-alike Unicode characters, encoded in DNS as punycode (xn--…) [9].

2.4 Explainability and Shortcut Learning

Model-agnostic explainers such as LIME [11] and SHAP [12] attribute a prediction to input features. For a linear model, the exact attribution is available directly from the coefficients. Geirhos et al. [13] describe *shortcut learning*: models exploit spurious correlations that hold in the benchmark but not in deployment, which in-distribution test scores cannot reveal.

2.5 Comparison and Research Gap

Table 2.1: Existing approaches compared with PhishLens

Approach	Strength	Limitation	PhishLens response
Blocklists [16]	Near-zero false positives	Blind to new URLs	Offline feature scoring
Black-box ML [3], [5]	High reported accuracy	No reasons; in-distribution tests	Linear model with exact contributions
Content analysis [18]	Uses page semantics	Must fetch hostile pages	Never fetches the URL
Squatting studies [6], [7]	Characterise abuse	Not packaged as analyst tools	Typo/combosquat signals

Approach	Strength	Limitation	PhishLens response
Cloud URL scanners	Convenient	Send URLs to third parties	Local; enrichment opt-in only

The gap addressed is the combination of (a) per-decision explainability, (b) an explicit, pre-registered out-of-template evaluation that exposes shortcut learning, and (c) a transparent hybrid fix whose effect is measured, all in a tool that is safe to run locally. The contribution is methodological and educational; the datasets are small and synthetic, and no claim is made about real-world detection rates.

CHAPTER 3

SYSTEM REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Endpoint
FR-1	Accept a URL or e-mail text (≤ 20,000 chars), extract URLs and score the first	POST /api/analyze
FR-2	Report risk (0–100), model risk, verdict, signals with severity, top feature contributions	POST /api/analyze
FR-3	Detect e-mail cues: urgency, credential/payment requests, generic greetings	POST /api/analyze
FR-4	Score up to 25 URLs in one batch with summary statistics	POST /api/analyze/batch
FR-5	Run WHOIS, DNS (A/MX/SPF/DMARC) and TLS checks only with an authorization flag	POST /api/enrich
FR-6	Expose held-out model metrics	GET /api/metrics
FR-7	Compare four classifiers with confusion matrices and cross-validation	GET /api/compare
FR-8	Provide safe and phishing example inputs; report tool availability	GET /api/examples, /api/tools
FR-9	Export the analysis as JSON or print it	Browser UI

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

ID	Category	Requirement	Verification
NFR-1	Safety	Analysed URLs are never fetched	Code review; no HTTP client in scoring path
NFR-2	Security	Enrichment cannot reach internal addresses (SSRF)	test_tls_check_refuses_…
NFR-3	Security	OWASP headers, input caps, rate limits	tests/test_security.py
NFR-4	Explainability	Every score lists signals and contributions	UI and API inspection
NFR-5	Performance	Single analysis p95 < 50 ms	scripts/loadtest.py
NFR-6	Reproducibility	Fixed seeds for data, split and models	make evaluate / compare / probes
NFR-7	Usability	Responsive to 320 px; 44 px targets	Mobile screenshots
NFR-8	Portability	One-command Docker start	docker compose up

3.3 Hardware and Software Requirements

Table 3.3: Hardware and software requirements

Item	Minimum	Used
CPU / RAM	64-bit CPU, 4 GB RAM	Apple Silicon, 8 GB+

Item	Minimum	Used
OS	Windows 10+, macOS 12+, Linux	macOS; container Debian
Runtime	Docker Desktop, or Python 3.9+	Python 3.12 in container
Libraries	Flask, scikit-learn, numpy, dnspython	requirements.txt
Tools	whois, dig, OpenSSL	Installed in image

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture

PhishLens separates feature extraction, modelling, evaluation and enrichment into independent modules behind a Flask API (Figure 4.1). The same `extract()` function produces training vectors and live vectors, which removes train/serve skew.

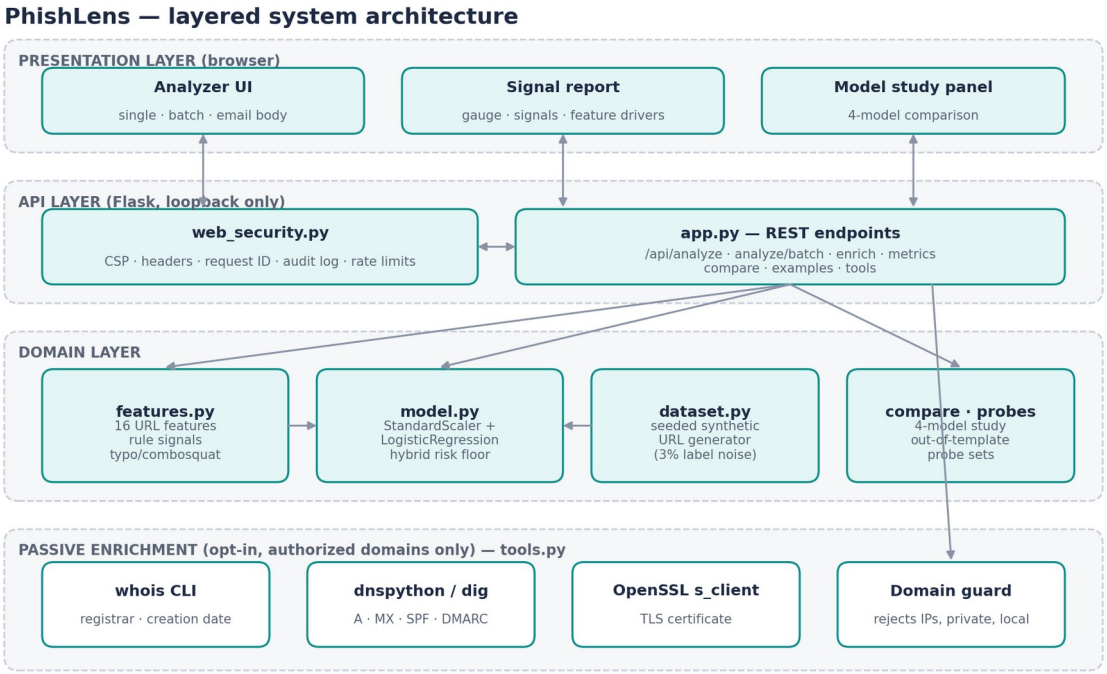


Figure 4.1: Layered architecture of PhishLens

4.2 Data Flow Diagrams

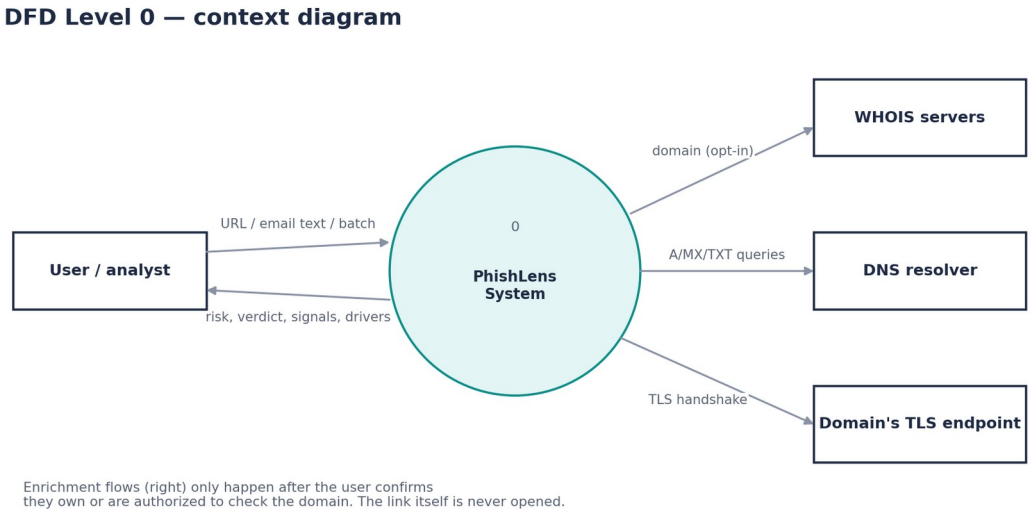


Figure 4.2: DFD level 0 (context diagram)

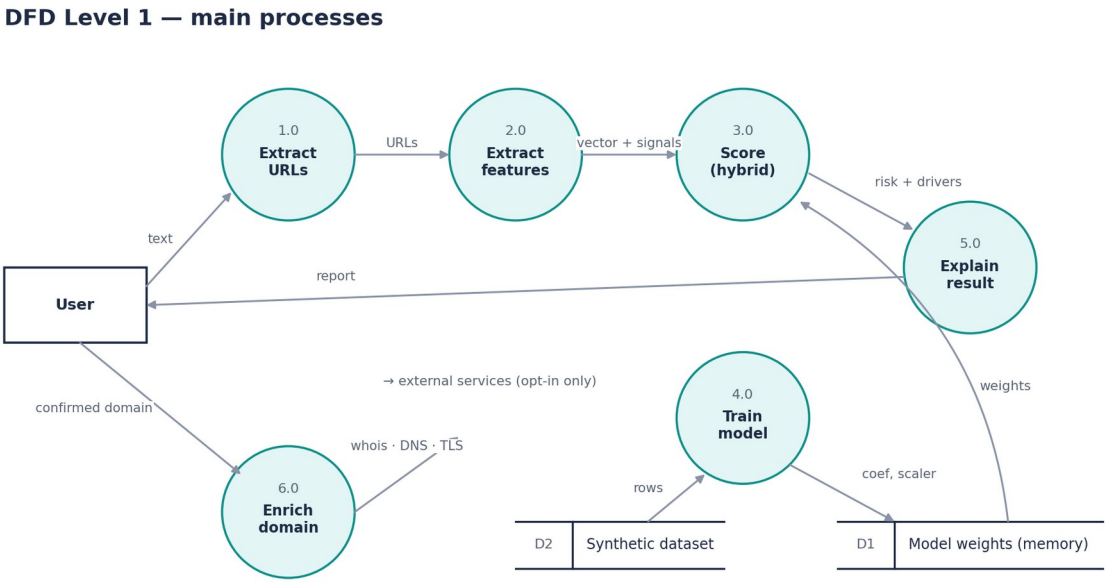


Figure 4.3: DFD level 1

4.3 UML Diagrams

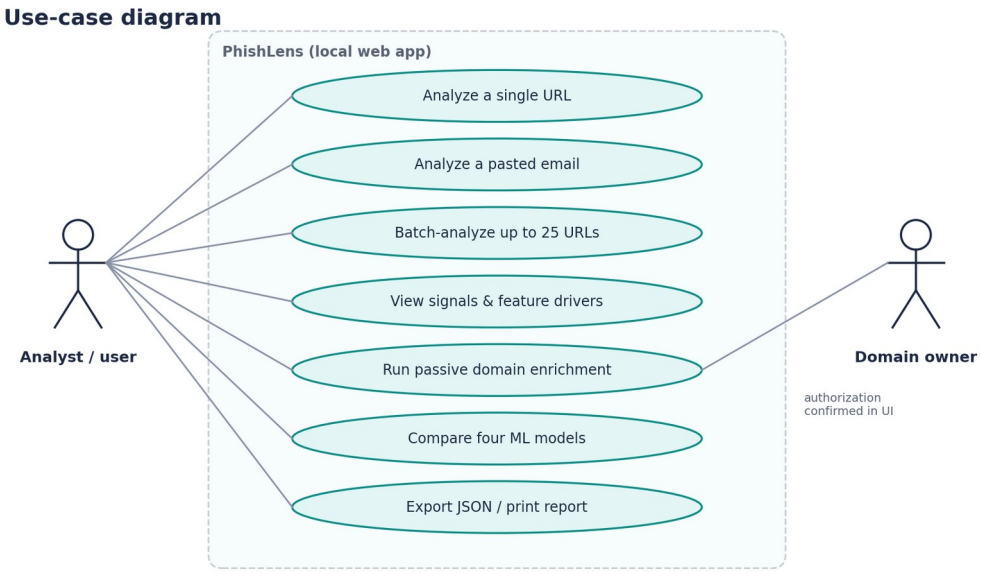


Figure 4.4: Use-case diagram

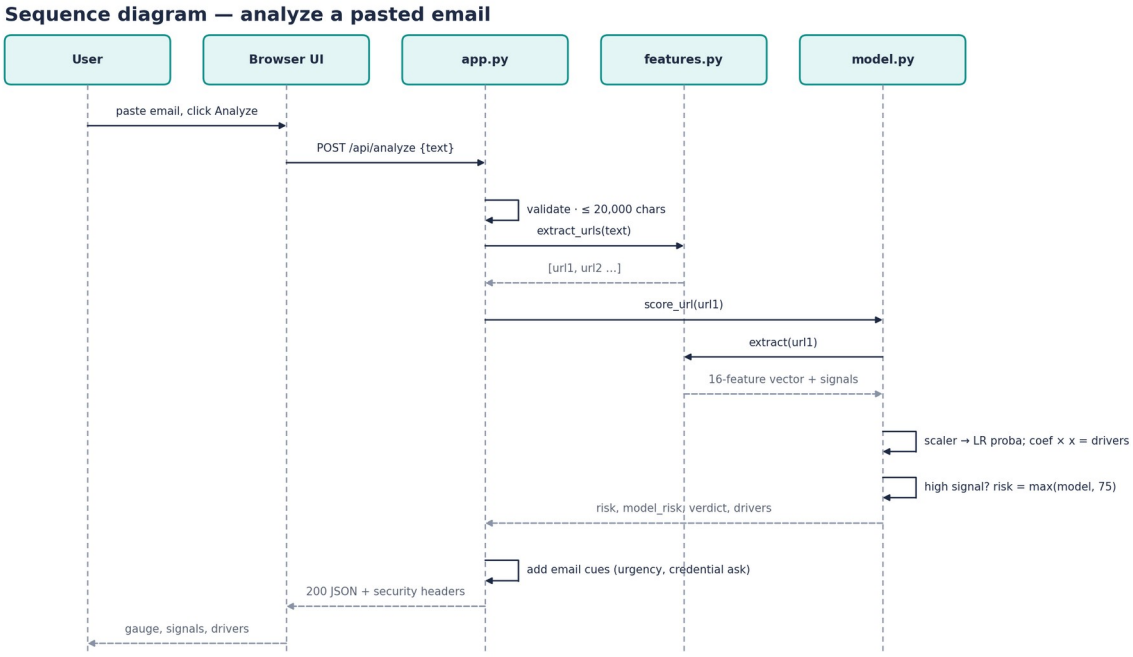


Figure 4.5: Sequence diagram for analysing a pasted e-mail

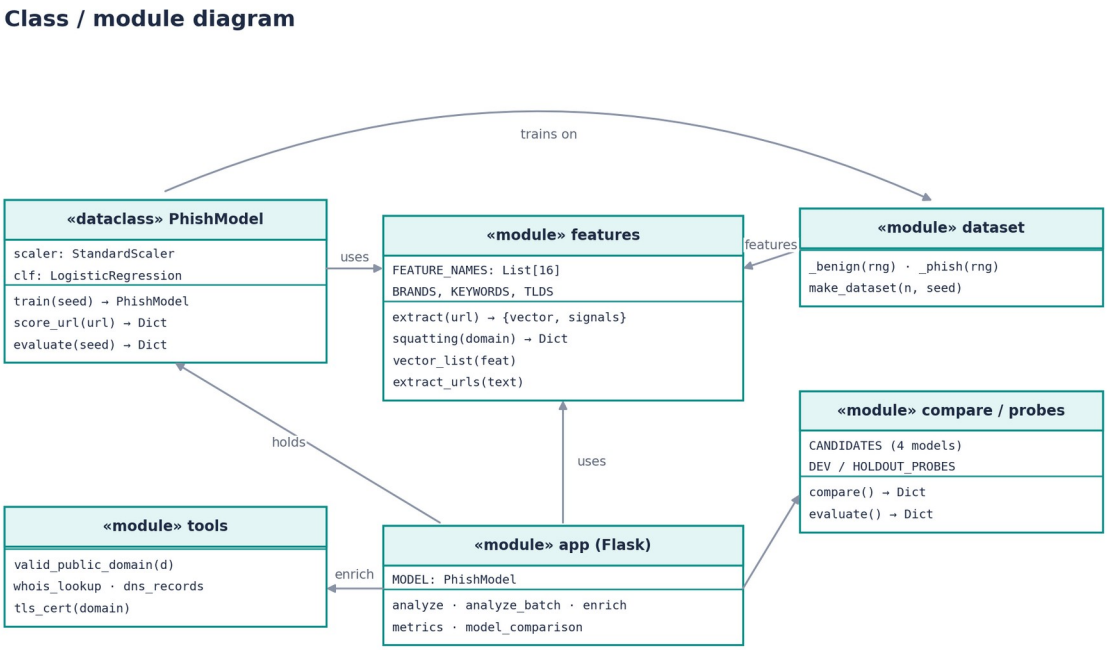


Figure 4.6: Class and module diagram

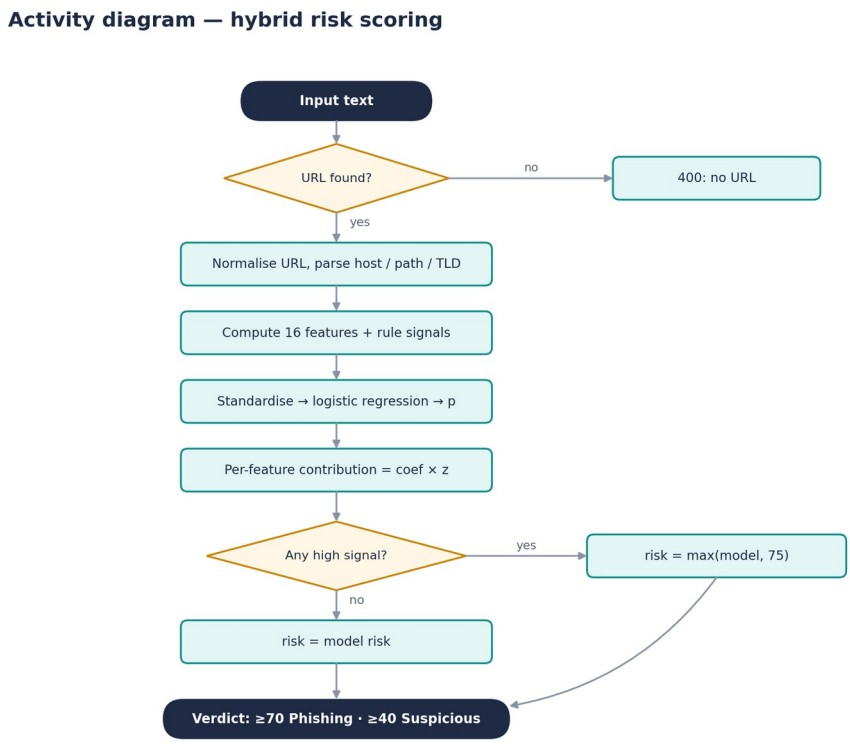


Figure 4.7: Activity diagram of hybrid risk scoring

4.4 Deployment

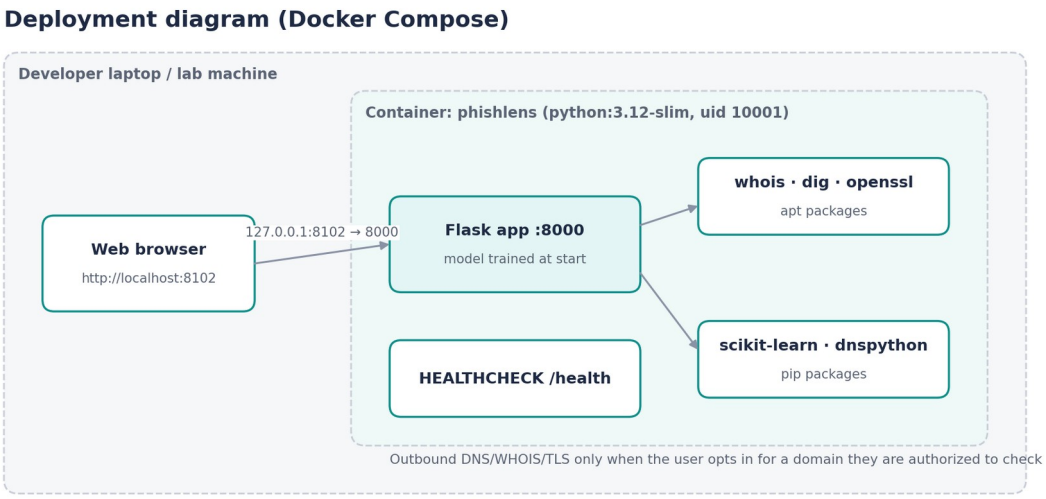


Figure 4.8: Deployment diagram

4.5 Feature Set

Table 4.1: The 16 model features

Feature	Meaning	Why it matters
url_length, host_length	Characters in URL / host	Long hosts hide the real domain
num_dots, num_hyphens, num_subdomains	Host structure	brand.com.evil.tld patterns
has_ip_host	Host is an IPv4 literal	Avoids registering a domain
has_at_symbol	'@' anywhere	Browser ignores text before '@'
is_punycode	xn-- label present	Homograph attacks [8], [9]
uses_https	Scheme is https	Weak signal; phishers use TLS too
suspicious_tld	TLD in a cheap/abused list	Correlates with abuse
is_shortener	Known shortener domain	Hides the destination
has_redirect	url=/redirect or // in path	Open-redirect abuse
digit_ratio_host, host_entropy	Randomness of host	Generated domains
brand_lookalike	Brand present but not the registered domain	Impersonation
suspicious_keywords	login, verify, account … (≤ 5)	Credential lures

CHAPTER 5

METHODOLOGY AND IMPLEMENTATION

5.1 Technology Stack

Table 5.1: Technology choices

Technology	Role	Justification
Python 3.12, Flask 3	API	Small, explicit routing; easy to audit
scikit-learn [10], numpy	Models and metrics	Standard, reproducible ML toolkit
Logistic regression	Deployed model	Exact per-feature attributions, millisecond training
dnspython, whois, OpenSSL	Passive enrichment	Standard protocols; no third-party API keys
Vanilla HTML/CSS/JS	UI	No build chain; strict CSP
Docker, pytest, GitHub Actions	Delivery and QA	Reproducible tools and tests

5.2 Stage 1 — Feature Extraction and Rule Signals

What: `extract(url)` normalises the URL, parses host, path and TLD, computes the feature vector and a list of (signal, severity) pairs. **Why:** the same interpretable features drive both the model and the explanation. **How:** the registered domain is approximated by the last two labels (a known simplification; see limitations).

5.3 Stage 2 — Synthetic Training Data

`make_dataset()` generates 2,400 URLs (half benign, half phishing) from templates for seven phishing styles: brand on a suspicious TLD, brand as a subdomain of junk, IP host, '@' trick, punycode, long hyphenated host and shortener. Benign URLs use brand and generic names on common TLDs, with some deliberately overlapping login paths. 3% of labels are flipped so the task cannot be solved perfectly. All seeds are fixed.

5.4 Stage 3 — Explainable Model

Features are standardised and a logistic-regression model is fitted. For a prediction, the log-odds decompose exactly into a sum of per-feature terms: **contribution_i = β_i · z_i, where z_i = (x_i − μ_i) / σ_i is the standardised feature value and β_i the learned coefficient**. The UI shows the six largest terms and whether each raises or lowers risk.

Listing 5.2: Scoring with contributions and the hybrid floor (phishlens/model.py)

```
    def score_url(self, url: str) -> Dict:
        feat = extract(url)
        x = np.array(vector_list(feat), dtype=float).reshape(1, -1)
```

```
        xs = self.scaler.transform(x)
        proba = float(self.clf.predict_proba(xs)[0, 1])
        # Per-feature contribution to the log-odds (coef * standardised value).
        contrib = (self.clf.coef_[0] * xs[0])
        order = np.argsort(-np.abs(contrib))
        drivers = [{
            "feature": FEATURE_NAMES[i],
            "value": feat["vector"][FEATURE_NAMES[i]],
            "contribution": round(float(contrib[i]), 3),
            "direction": "raises" if contrib[i] > 0 else "lowers",
        } for i in order[:6] if abs(contrib[i]) > 0.01]
        model_risk = round(proba * 100, 1)
        # Hybrid scoring: a high-severity rule signal (IP host, punycode, '@'
        # trick, brand on a foreign domain, typosquat, combosquat) is strong
        # evidence on its own, so it sets a floor under the learned score. The
        # out-of-template probes in probes.py showed the model alone missing
        # these whenever the rest of the URL looked ordinary.
        high = [t for t, sev in feat["signals"] if sev == "high"]
        risk = max(model_risk, HIGH_SIGNAL_FLOOR) if high else model_risk
        verdict = "Phishing" if risk >= 70 else ("Suspicious" if risk >= 40 else "Likely
safe")
        return {
            "url": url, "host": feat["host"], "registered_domain": feat["registered_domain"],
            "risk": risk, "model_risk": model_risk, "verdict": verdict,
            "floor_applied": bool(high) and risk > model_risk,
            "signals": [{"text": t, "severity": s} for t, s in feat["signals"]],
            "drivers": drivers,
            "features": feat["vector"],
        }
```

5.5 Stage 4 — Model Comparison

`compare.py` trains four candidates on identical data (train seed 5) and evaluates them on an independent set (seed 51) with accuracy, precision, recall, F1, ROC-AUC, confusion matrices and 5-fold stratified cross-validation.

5.6 Stage 5 — Out-of-Template Probes (Pre-Registered Hold-Out)

To test generalisation, two probe sets of hand-written URLs were created with tricks the generator never produces. The **development set** (10 phishing, 10 benign) was written first and used for error analysis. The **hold-out set** (10 phishing, 12 benign, including awkward benign cases such as a Wikipedia article about PayPal) was written and fixed *before* designing any fix and was never used for tuning. A URL is counted as flagged when its score is at least 40 (Suspicious or Phishing).

5.7 Stage 6 — Typosquat, Combosquat and the Hybrid Floor

Error analysis on the development set showed that the model ignored strong evidence when the rest of a URL looked ordinary, for example punycode on a .com domain. Two rule signals were added. A **typosquat** match is a registered-domain label that becomes a brand after undoing digit look-alikes (0→o, 1→l, …), or is one edit away from a brand of six or more letters. A **combosquat** match is a hyphenated label containing a brand [6]. A brand mentioned only in the URL path was downgraded to medium severity because it is weak evidence. The

hybrid rule then sets risk = max(model risk, 75) whenever a high-severity signal is present. The model's own score is still reported as `model_risk`, so explainability is preserved.

Listing 5.3: Squatting detection (phishlens/features.py)

```
def squatting(registered: str) -> Dict[str, List[str]]:
    """Brand-squatting checks on the registered domain (e.g. 'paypa1-login.com').

    * typosquat: a label is a brand after undoing digit look-alikes (paypa1,
      amaz0n) or is one edit away from a brand of 6+ letters (linkedln).
    * combosquat: the label joins a brand with other words using hyphens
      (hdfc-netbanking-kyc-update), the pattern studied by Kintis et al. (2017).
    The brand's own domain (label exactly equal to the brand) never matches.
    """
    label = registered.rsplit(".", 1)[0] if "." in registered else registered
    typo, combo = set(), set()
    if label in BRANDS:
        return {"typosquat": [], "combosquat": []}
    parts = label.split("-")
    for part in parts:
        norm = part.translate(_HOMOGLYPHS)
        for brand in BRANDS:
            if norm == brand and part != brand:
                typo.add(brand)
            elif len(brand) >= 6 and _edit_distance_one(norm, brand):
                typo.add(brand)
            if len(parts) > 1 and (norm == brand or (len(brand) >= 5 and
norm.startswith(brand))):
                combo.add(brand)
    return {"typosquat": sorted(typo), "combosquat": sorted(combo)}
```

5.8 Stage 7 — Safe Passive Enrichment

Enrichment requires an explicit `confirm_own_domain` flag and a valid public DNS name (IP literals and local names are rejected). During this project a weakness was found: a public-looking name could resolve to 127.0.0.1, a private address or the cloud metadata address, and the TLS check would then connect to it, a server-side request forgery (SSRF) [17]. The fix resolves the name once, refuses any non-global address, and connects OpenSSL to the vetted IP with SNI set to the name, so DNS rebinding between two lookups cannot change the target.

Listing 5.4: SSRF-safe resolution (phishlens/tools.py)

```
def resolve_public(host: str):
    """Resolve host once and return one public IPv4 address, or None.

    A public-looking *name* can still point at 127.0.0.1, a private range or a
    cloud metadata address (and DNS rebinding can change the answer between two
    lookups). Refusing any non-global answer and then connecting to the vetted
    IP prevents the TLS check from being turned into a request to internal
    services (SSRF).
    """
    try:
        infos = socket.getaddrinfo(host, None, socket.AF_INET, socket.SOCK_STREAM)
    except (socket.gaierror, UnicodeError, OSError):
        return None
    addresses = {info[4][0] for info in infos}
    if not addresses:
        return None
    for address in addresses:
        if not ipaddress.ip_address(address).is_global:
            return None
    return sorted(addresses)[0]
```

5.9 Stage 8 — Hardening, Logging and Performance

The shared `web_security` layer adds CSP and OWASP headers, request IDs, body-free access logs, JSON errors and rate limits (enrichment 10/min, comparison 6/min); every authorized enrichment is written to an audit log line. Load testing revealed that `/api/metrics` recomputed the held-out evaluation on every request (about 240 ms); it is now computed once and cached.

5.10 Secure Coding Practices Applied

- Never fetch analysed URLs; treat all input as hostile text.
- Escape every URL fragment before rendering; strict CSP.
- Gate network enrichment behind explicit authorization; validate names; refuse non-global resolutions (SSRF).
- Call whois and OpenSSL with argument lists and timeouts.
- Bound input sizes and batch lengths; rate-limit network and CPU-heavy endpoints.
- Log request metadata and enrichment audit events, never bodies.

CHAPTER 6

TESTING AND VALIDATION

6.1 Strategy

Table 6.1: Test levels

Level	Scope	Tool
Unit	features, model, squatting, comparison, TLS parser	pytest
Integration	HTTP endpoints through Flask's test client	pytest
Security	Authorization gate, domain validation, SSRF, headers, rate limits	pytest + monkeypatched DNS
Model validation	In-distribution test set, 5-fold CV, out-of-template probes	compare.py, probes.py
Functional / UI	Browser flows on the Docker build	Playwright + screenshots
Performance	Throughput and latency	scripts/loadtest.py

6.2 Automated Test Cases

The suite of 18 tests ran with **18 passed, 0 failed**.

Table 6.2: Automated test cases and results

ID	Type	Scenario	Expected	Actual (test function)
TC-01	Unit	Feature extraction of '@' trick URL	has_at_symbol = 1; 16 features	**PASS** (test_feature_extraction)
TC-02	Unit	URL extraction from e-mail body	paypal-verify.tk URL found	**PASS** (test_extract_urls_from_email)
TC-03	Unit	Model quality and sanity	AUC ≥ 0.9; google.com < 25; phishing > 70	**PASS** (test_model_quality_and_scoring)
TC-04	Integration	Analyse endpoint	Phishing/Suspicious verdict with drivers	**PASS** (test_api)
TC-05	Integration	Batch endpoint and limits	Summary stats; > 25 items rejected	**PASS** (test_batch_endpoint_and_input_limits)
TC-06	Security	Enrichment requires authorization	400 without confirmation flag	**PASS** (test_live_enrichment_requires_owner_confirmation_and_public_domain)
TC-07	Security	Domain validation	IPs, localhost, malformed names rejected	**PASS** (test_domain_guard_rejects_local_and_ip_targets)
TC-08	Unit	TLS certificate parsing	Issuer and dates parsed	**PASS** (test_openssl_certificate_output_parser)

ID	Type	Scenario	Expected	Actual (test function)
TC-09	Unit	Model comparison	4 models; confusion sums = n; AUC ≥ 0.9	**PASS** (test_model_comparison_reports_every_candidate)
TC-10	Unit	Squatting signals	paypa1, linkedln, hdfc-… detected; own domains not	**PASS** (test_squatting_signals)
TC-11	Unit	Hybrid floor	Punycode raised ≥ 70; path mention stays < 40	**PASS** (test_hybrid_floor_catches_punycode_but_not_path_mentions)
TC-12	Validation	Hold-out probes	Hybrid recall > model recall; 0 FP	**PASS** (test_holdout_probes_hybrid_beats_model_only)
TC-13	Security	SSRF: names resolving to internal IPs	Loopback, metadata, private refused	**PASS** (test_tls_check_refuses_names_that_resolve_to_internal_addresses)
TC-14	Security	Security headers	CSP, DENY, nosniff, no-referrer	**PASS** (test_security_headers_on_page_and_api)
TC-15	Security	No caching of API responses	Cache-Control: no-store	**PASS** (test_api_responses_are_not_cached)
TC-16	Security	Request-ID log-injection guard	Forged ID replaced	**PASS** (test_request_id_is_echoed_only_when_safe)
TC-17	Security	Unknown API route	JSON 404	**PASS** (test_unknown_api_route_returns_json_error)
TC-18	Security	Rate limit on comparison	7th call → 429	**PASS** (test_expensive_endpoint_is_rate_limited)

6.3 Functional Checks on Sample URLs

Table 6.3 shows live results from the Docker build for the example inputs and two probes. The final row is the punycode homograph that the model alone misses and the hybrid floor catches.

Table 6.3: Live scoring of sample inputs (Docker build)

Input URL	Model risk	Final risk	Verdict
https://www.google.com/	3.3	3.3	Likely safe
https://github.com/explore	3.5	3.5	Likely safe
https://www.amazon.in/orders	3.8	3.8	Likely safe
http://paypal-verify.tk/login	96.1	96.1	Phishing
http://apple.com.secure99.ml/confirm	98.3	98.3	Phishing

Input URL	Model risk	Final risk	Verdict
https://netflix.com@account-update.xyz/billing	97.1	97.1	Phishing
http://192.0.2.44/secure/verify	97.5	97.5	Phishing
http://bit.ly/hdfc482	96.9	96.9	Phishing
http://paypa1-secure-login.com/verify	4.7	75.0	Phishing
http://xn--pypal-4ve.com/signin	4.3	75.0	Phishing

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Working System

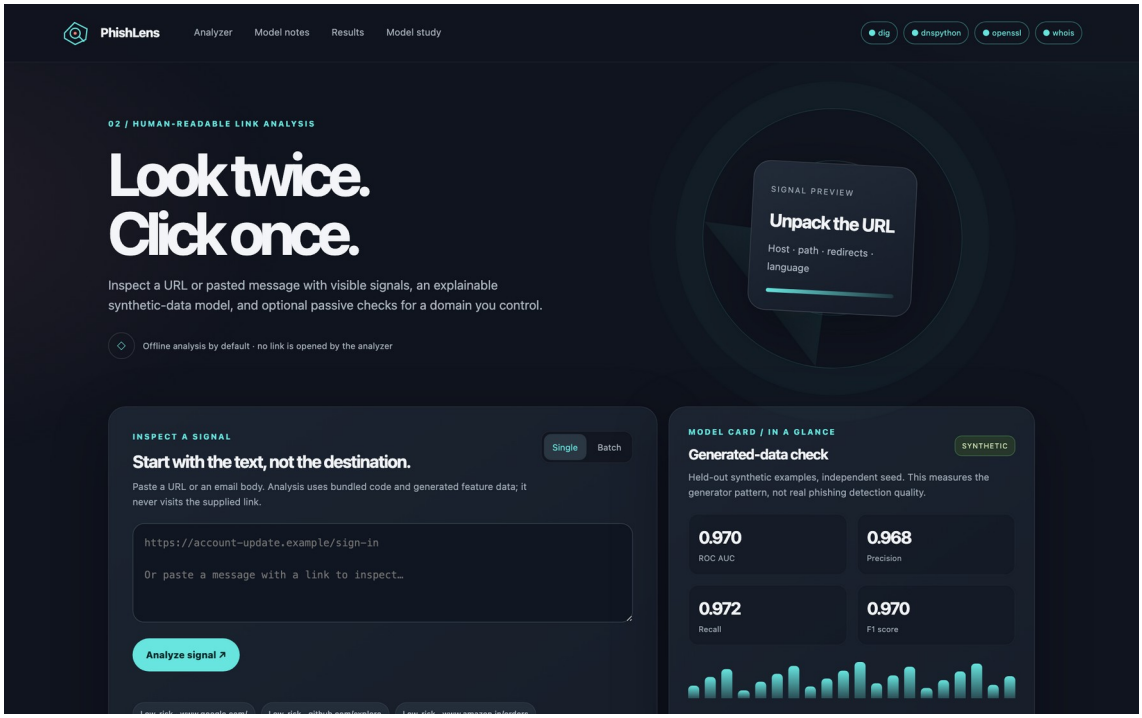


Figure 7.1: PhishLens analyser

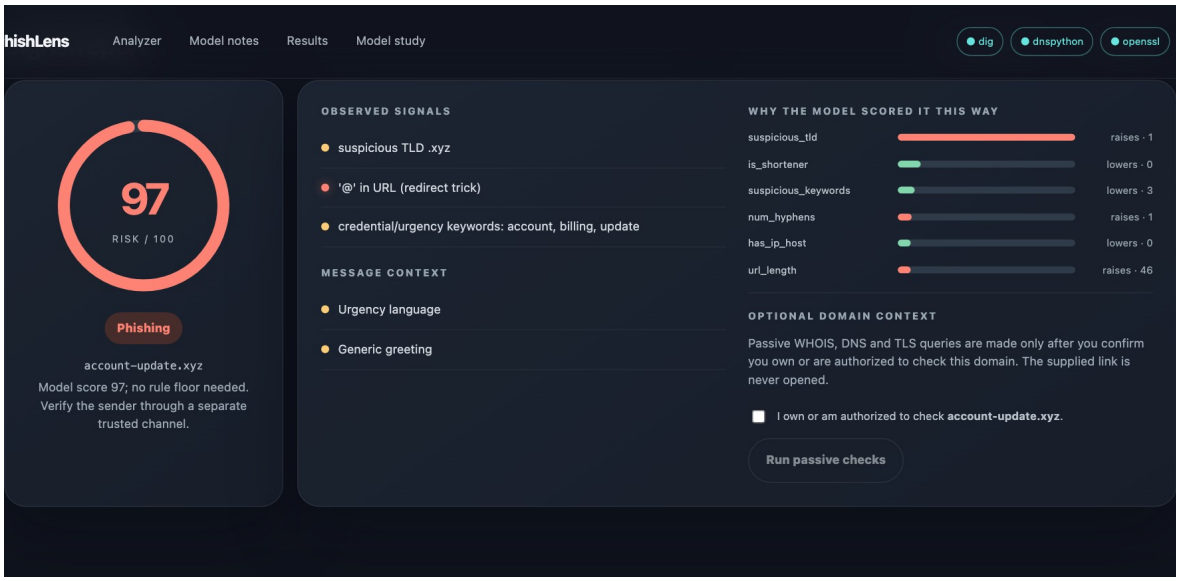


Figure 7.2: Phishing verdict for a pasted e-mail with signals and feature drivers

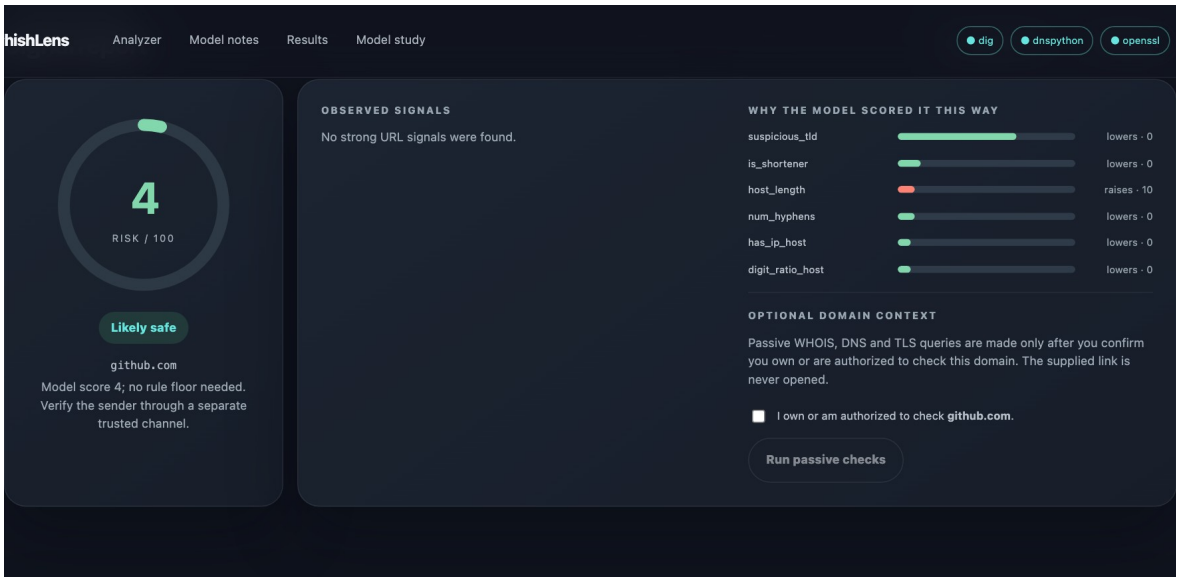


Figure 7.3: Low-risk verdict for a legitimate URL

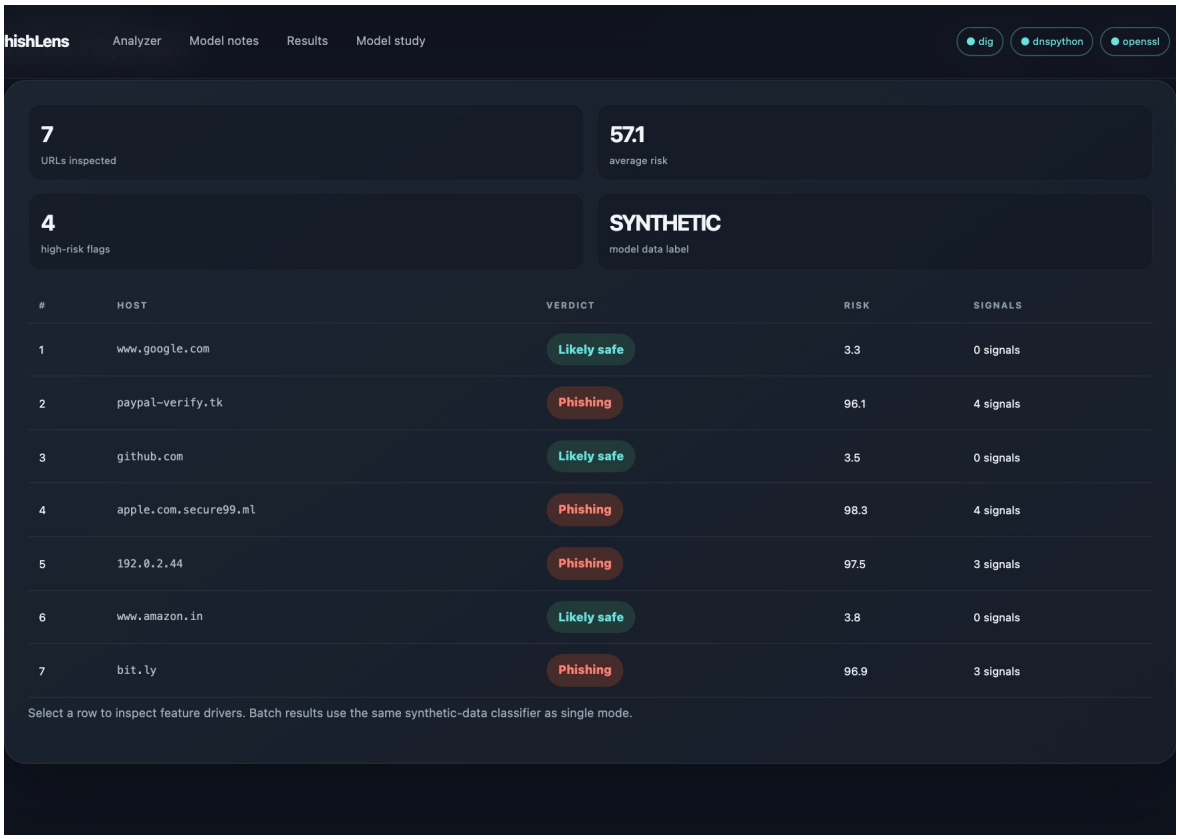


Figure 7.4: Batch analysis of seven URLs

7.2 In-Distribution Model Comparison

Table 7.1: Model comparison (train n = 2,400, test n = 2,400)

Model	Acc.	Prec.	Rec.	F1	AUC	5-fold AUC	Train ms
Logistic Regression	0.970	0.968	0.972	0.970	0.968	0.970 ± 0.007	3.4
Decision Tree	0.969	0.967	0.971	0.969	0.966	0.963 ± 0.008	1.6
Random Forest	0.962	0.961	0.962	0.962	0.964	0.973 ± 0.008	126.7

Model	Acc.	Prec.	Rec.	F1	AUC	5-fold AUC	Train ms
Gaussian Naive Bayes	0.970	0.968	0.972	0.970	0.969	0.970 ± 0.005	0.6

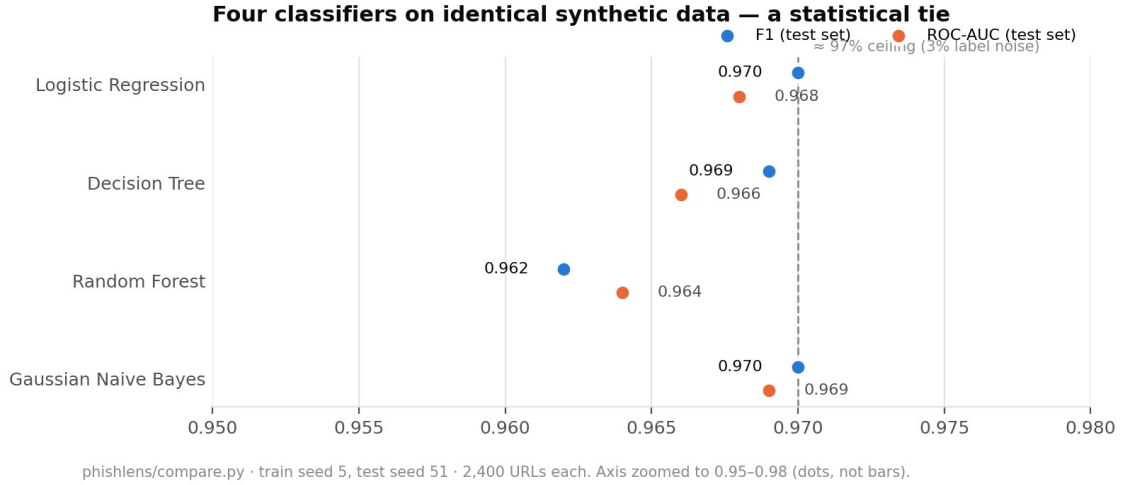


Figure 7.5: F1 and ROC-AUC per model (axis zoomed)

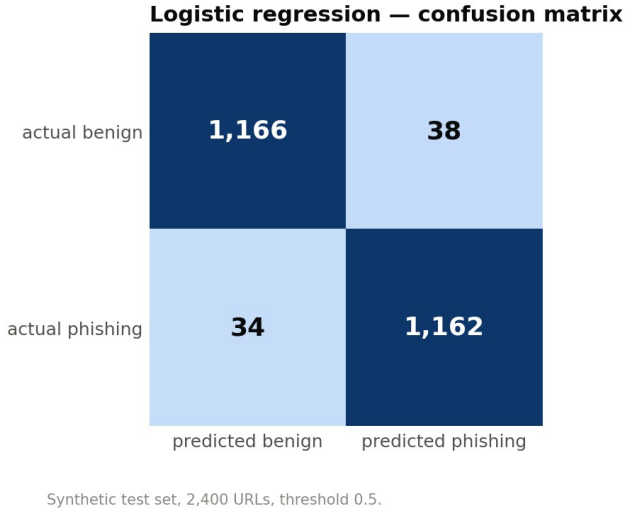


Figure 7.6: Confusion matrix of the deployed logistic-regression model

All four models fall within about one percentage point (F1 0.962–0.970), essentially at the 97% ceiling implied by 3% label noise. The logistic regression's 38 false positives and 34 false negatives are of the same order as the 72 flipped labels in the test set. Since accuracy ties, the deciding criterion is explanation: logistic regression gives exact per-feature attributions, and is the deployed model. These numbers describe the generator, not real phishing.

7.3 Out-of-Template Generalisation

Table 7.2: Probe results (flagged = score ≥ 40)

Set and scorer	TP	FN	FP	TN	Prec.	Recall	F1
Dev — model only	1	9	0	10	1.00	0.10	0.18
Dev — hybrid	10	0	0	10	1.00	1.00	1.00
Hold-out — model only	1	9	0	12	1.00	0.10	0.18
Hold-out — hybrid	9	1	0	12	1.00	0.90	0.95

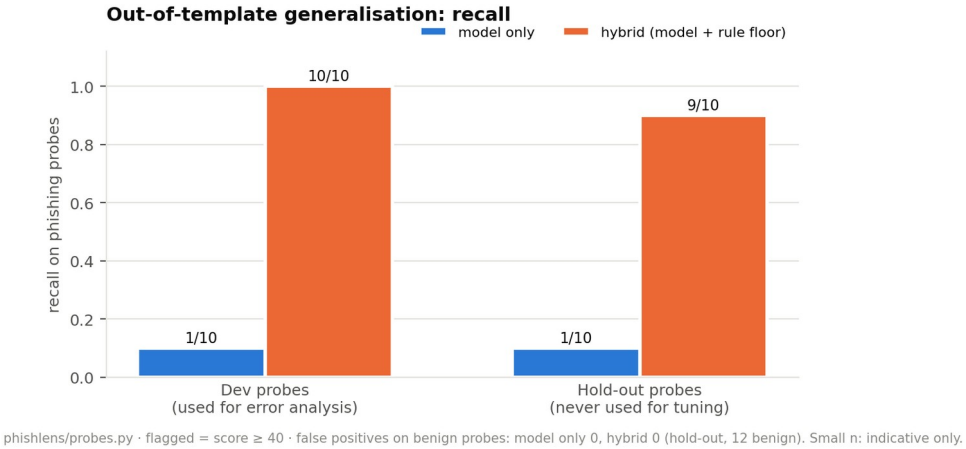


Figure 7.7: Recall on out-of-template phishing probes

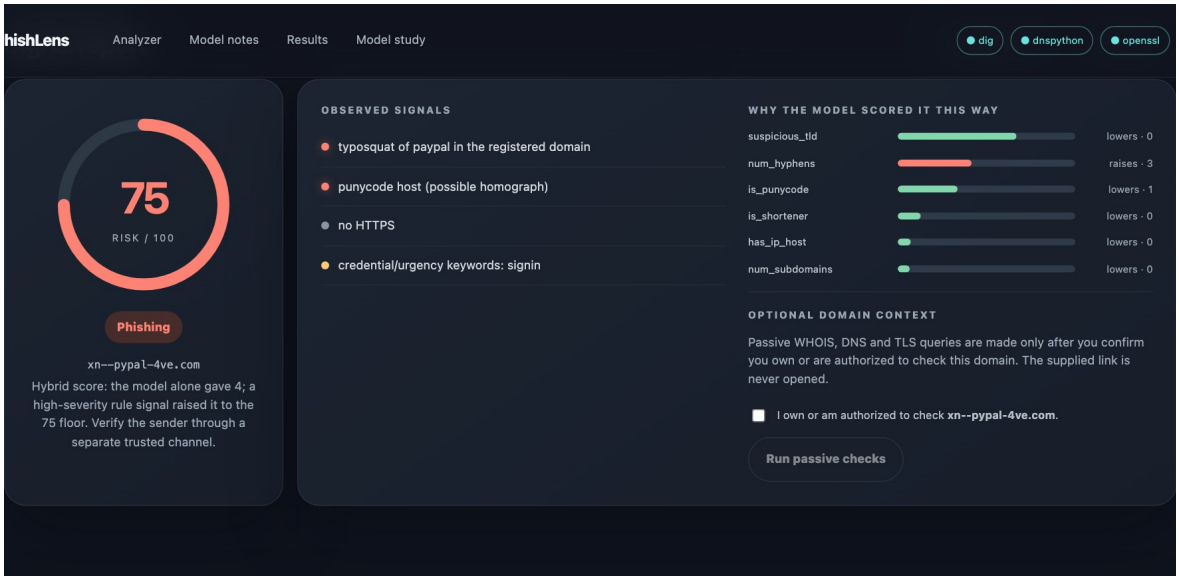


Figure 7.8: Punycode homograph: model risk is low, the hybrid floor raises the verdict

Finding 1 — shortcut learning. A model with 97% in-distribution F1 caught only 1 of 10 hold-out phishing probes (the IP-host URL). The generator always pairs tricks like punycode with suspicious TLDs and plain HTTP, so the model learned to rely on those correlates rather than on the trick itself [13].

Finding 2 — hybrid scoring generalises better without false alarms. On the hold-out set, which was fixed before the hybrid was designed, recall rose from 0.1 to 0.9 with 0 false

positives among 12 benign URLs, including two benign pages that mention PayPal in their path. The development-set result (10/10) is optimistic by construction because it guided the design.

Remaining miss: `http://secure-account-update.weebly.com/` (risk 3.3). A brandless lure on a free web-hosting subdomain contains no high-severity signal; detecting it needs hosting-provider and content features. **Caveat:** with 10 phishing and 12 benign hold-out URLs, a 95% confidence interval for recall is wide (roughly 0.55–1.0 for 9/10); the result is indicative, not conclusive.

7.4 Security Finding: SSRF in Enrichment

The SSRF weakness described in Section 5.7 was found by reviewing the enrichment path against the OWASP SSRF guidance [17]. It was fixed in code, covered by TC-13 (which simulates DNS answers of 127.0.0.1, 169.254.169.254 and 10.1.2.3), and documented in SECURITY.md and the changelog.

7.5 Performance

Table 7.3: Load-test results (200 requests, 8 workers)

Endpoint	Req/s	p50 ms	p95 ms	max ms	Errors
single URL analysis	1,391	5.3	9.3	10.6	0
email body analysis	1,576	5.1	6.9	8.0	0
batch of 10 URLs	656	11.9	17.3	21.7	0
model metrics	2,033	3.8	5.3	6.6	0

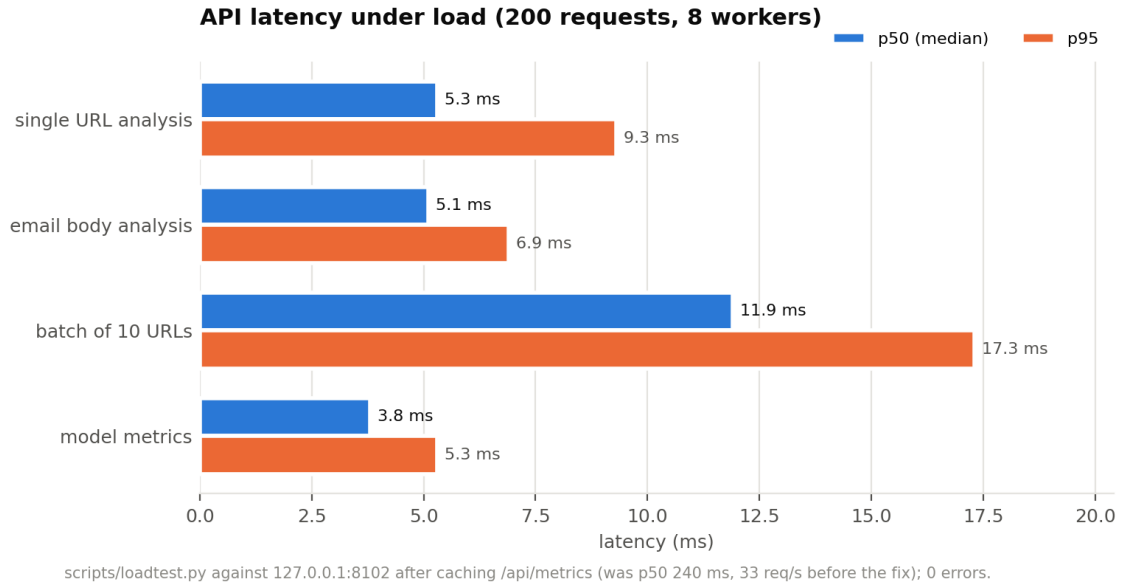


Figure 7.9: Latency per endpoint

Before caching, `/api/metrics` ran at 33 requests/s with a 240 ms median, because it re-scored the 2,400-URL test set each time. After caching it reached about 2,000 requests/s with a median under 4 ms, an example of performance testing changing the design.

7.6 Mobile Layout

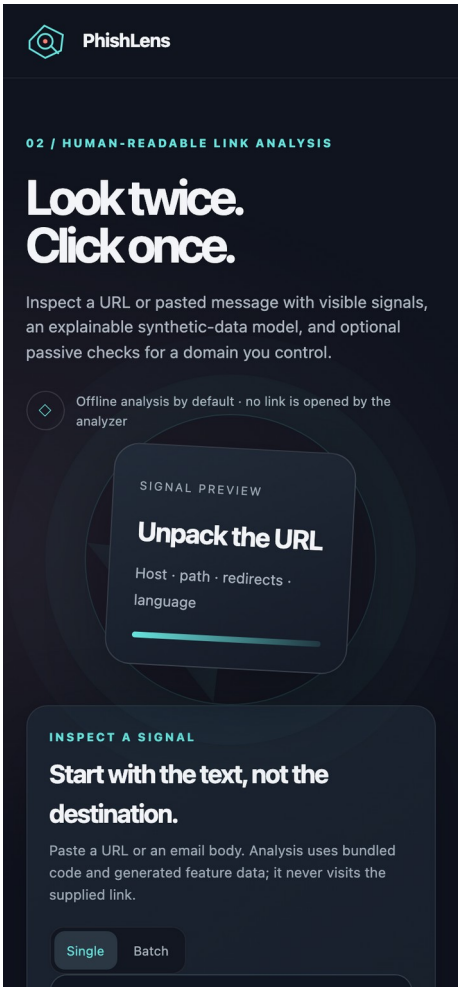


Figure 7.10: Layout at 390 px width

7.7 Limitations

- Training data are synthetic; no claim is made about real-world accuracy.
- Probe sets are small (42 URLs in total); confidence intervals are wide.
- The registered domain uses the last two labels, which mishandles suffixes such as .co.uk; the Public Suffix List would fix this.
- Brandless lures on free hosting are missed; page-content features are out of scope by design.
- The brand list is fixed (20 brands); unseen brands are not recognised by squatting rules.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

PhishLens delivers an explainable, local phishing analyser and, more importantly, an honest account of how it generalises. In-distribution scores of about 97% hid a model that caught 1 in 10 phishing URLs built with unseen tricks. A transparent hybrid of model and rule signals raised hold-out recall to 9 in 10 without false positives, while keeping every verdict explainable. The project also found and fixed an SSRF weakness and a performance issue through testing. All 18 automated tests pass.

This second design project builds directly on the foundations of the first (CryptoGuard): safe tool integration and honest measurement. It adds machine learning, evaluation design and threat modelling of network-facing features.

8.2 Future Scope

- Train and evaluate on a licensed public phishing corpus with a time-based split.
- Use the Public Suffix List for registered domains.
- Learn the brand list from certificate-transparency logs; add visual-similarity checks for homographs.
- Calibrate probabilities and report confidence intervals automatically.
- Browser-extension front end that scores links before a click, fully offline.

REFERENCES

[1] Anti-Phishing Working Group (APWG), “Phishing Activity Trends Report,” quarterly reports. [Online]. Available: https://apwg.org/trendsreports/

[2] Verizon, “Data Breach Investigations Report (DBIR),” 2024. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[3] J. Ma, L. K. Saul, S. Savage and G. M. Voelker, “Beyond blacklists: learning to detect malicious web sites from suspicious URLs,” in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2009, pp. 1245–1254.

[4] S. Garera, N. Provos, M. Chew and A. D. Rubin, “A framework for detection and measurement of phishing attacks,” in Proc. ACM Workshop on Recurring Malcode (WORM), 2007, pp. 1–8.

[5] O. K. Sahingoz, E. Buber, O. Demir and B. Diri, “Machine learning based phishing detection from URLs,” Expert Systems with Applications, vol. 117, pp. 345–357, 2019.

[6] P. Kintis, N. Miramirkhani, C. Lever, Y. Chen, R. Romero-Gómez, N. Pitropakis, N. Nikiforakis and M. Antonakakis, “Hiding in plain sight: a longitudinal study of combosquatting abuse,” in Proc. ACM CCS, 2017, pp. 569–586.

[7] J. Szurdi, B. Kocso, G. Cseh, J. Spring, M. Felegyhazi and C. Kanich, “The long ‘taile’ of typosquatting domain names,” in Proc. 23rd USENIX Security Symposium, 2014, pp. 191–206.

[8] E. Gabrilovich and A. Gontmakher, “The homograph attack,” Communications of the ACM, vol. 45, no. 2, p. 128, 2002.

[9] A. Costello, “Punycode: A Bootstring encoding of Unicode for Internationalized Domain Names in Applications (IDNA),” IETF RFC 3492, 2003.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] M. T. Ribeiro, S. Singh and C. Guestrin, “‘Why should I trust you?’: explaining the predictions of any classifier,” in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[12] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017.

[13] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge and F. A. Wichmann, “Shortcut learning in deep neural networks,” Nature Machine Intelligence, vol. 2, pp. 665–673, 2020.

[14] S. Kitterman, “Sender Policy Framework (SPF) for authorizing use of domains in email, version 1,” IETF RFC 7208, 2014.

[15] M. Kucherawy and E. Zwicky, “Domain-based Message Authentication, Reporting, and Conformance (DMARC),” IETF RFC 7489, 2015.

[16] Google, “Safe Browsing.” [Online]. Available: https://developers.google.com/safe-browsing

[17] OWASP Foundation, “Server-Side Request Forgery Prevention Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/

[18] Y. Zhang, J. I. Hong and L. F. Cranor, “CANTINA: a content-based approach to detecting phishing web sites,” in Proc. 16th Int. World Wide Web Conference (WWW), 2007, pp. 639–648.

APPENDIX A: INSTALLATION AND USER MANUAL

8. Install and start Docker Desktop.
9. In the project folder run `docker compose up --build -d`.
10. Open http://localhost:8102.
11. Paste a URL or e-mail and click Analyze; switch to Batch for up to 25 URLs.
12. Enrichment: tick the authorization box only for a domain you own or may check.

Listing A.1: Developer commands

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
make test        # 18 tests
make evaluate    # held-out metrics
make compare     # four-model study
make probes      # out-of-template evaluation
make run         # http://127.0.0.1:8000
python scripts/loadtest.py
```

Table A.2: Troubleshooting

Symptom	Fix
Enrichment returns 400	Tick the authorization checkbox; use a public DNS name
TLS check refused	The name resolves to a non-public address (SSRF guard working)
429 responses	Rate limit reached; wait a minute

APPENDIX B: DEMONSTRATION SCRIPT

13. Analyse https://github.com/explore: low risk, few signals.
14. Paste the suspended-account e-mail: high risk; read the signals and the top feature drivers; point to e-mail cues.
15. Batch-analyse the seven sample URLs.
16. Open Model study: four models tie at ~97% — explain the label-noise ceiling.
17. Analyse http://xn--pypal-4ve.com/signin: model risk ≈ 4, hybrid floor raises it; explain shortcut learning and the hold-out result 1/10 → 9/10.
18. Describe the SSRF fix and show the test.

APPENDIX C: API REFERENCE

Table C.1: REST endpoints

Method and path	Input	Output
POST /api/analyze	{text}	risk, model_risk, verdict, signals, drivers, features, email_signals
POST /api/analyze/batch	{items: [≤ 25]}	results[], count, average_risk, high_risk
POST /api/enrich	{domain, confirm_own_domain}	whois, dns, tls
GET /api/metrics	—	roc_auc, precision, recall, f1, n
GET /api/compare	—	results[] per model
GET /api/examples, /api/tools	—	examples; tool availability

APPENDIX D: REPOSITORY STRUCTURE

Listing D.1: Repository layout

```
├── .dockerignore
├── .env.example
├── .github/
│   └── workflows/
│       └── ci.yml
├── .gitignore
├── CHANGELOG.md
├── CONTRIBUTING.md
├── Dockerfile
├── LICENSE
├── Makefile
├── PROJECT_OVERVIEW.md
├── README.md
├── SECURITY.md
├── Start.command
├── docker-compose.yml
├── docs/
│   ├── README.md
│   ├── diagrams/  (12 images)
│   ├── index.html
│   └── screenshots/  (7 images)
├── phishlens/
│   ├── __init__.py
│   ├── app.py
│   ├── compare.py
│   ├── data/
│   ├── dataset.py
│   ├── evaluate.py
│   ├── features.py
│   ├── model.py
│   ├── probes.py
│   ├── static/
│   │   └── index.html
│   ├── tools.py
│   └── web_security.py
├── requirements.txt
├── scripts/
│   └── loadtest.py
└── tests/
    ├── test_phishlens.py
    └── test_security.py
```